Project Development Phase
ModelPerformance Test

Project Name	Implement Client Script & UI Policy (Incident)
Date	13 October 2026
Team ID	6ab4a5100fc666a751b4361b
Maximum Marks	

Model Performance Testing:

Project team shall fill the following information in model performance testing template.

S.No.	Parameter	Values	Screenshot
1	Project Content	~~Implement Client Script & UI Policy (Incident)~~ **Problem Statement** Incident records often require consistent and accurate data entry to ensure effective triage, routing, and resolution. However, relying solely on user awareness and manual checks can lead to incomplete, inconsistent, or incorrect data being submitted. This can impact reporting accuracy, SLA compliance, and overall service quality. To address this challenge, there is a need to enforce conditional field behavior and validation directly at the user interface level. Implementing standardized UI Policies and Client Scripts helps control field visibility, mandatory status, and save-time validation, ensuring that Incident records meet defined business rules before they are submitted. **Objective** The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how ServiceNow client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make fields mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met. The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform. **Skills** Incident Management UI Policy UI Policy Actions Client Scripts Form Validation **Task1: CreateUI PolicyonIncident** **Activity 1:Create UIPolicy** 1. Log in to the ServiceNow instance with administrative or configuration access.	

S.No.	Parameter	Values	Screenshot
2	Project Content (continued)	2. Navigate to: System UI → UI Policies 3. Click New to create a new UI Policy record. 4. Fill in the following details: ■ Name: High Impact Control This name clearly represents the purpose of the policy, which is to apply additional controls when an Incident has a high impact. ■ Table: Incident Specifies that the UI Policy applies to Incident records. ■ Active: true Enables the UI Policy so it is enforced on the Incident form. 5. Configure the Conditions section as follows: ■ Field: Impact ■ Operator: is ■ Value: 1 – High 6. This condition ensures the UI Policy is triggered only when the Incident impact is set to High. 7. And Ui policy action Assignment group 8. Check mandatory 9. Set Reverse if false to true. This ensures that when the condition is not met, any field changes applied by the policy are automatically reverted. 10.Review the configuration to ensure accuracy. 11.Click Submit to save the UI Policy.	

S.No.	Parameter	Values	Screenshot
3	Project Content (continued)		

S.No.	Parameter	Values	Screenshot
4	Project Content (continued)	**Task 2: Create UI Policy Action – Urgency** **Activity 1: Configure the Urgency Field** 1. Open the UI Policy Actions Go to the High Impact Control UI Policy you created earlier. Scroll down to the UI Policy Actions related list. This is where you define the specific behavior of fields when the policy conditions are met. 2. Create a New Action Click New to create a UI Policy Action. This lets you configure how individual fields behave when the UI Policy is triggered. 3. Fill in the Action Details ■ Field name: Urgency Select the Urgency field on the Incident form. This is the field you want to control. ■ Read-only: true Check this option to make the Urgency field read-only whenever the UI Policy condition is met (Impact = High). Users will be able to see the field but cannot change its value.	

S.No.	Parameter	Values	Screenshot
5	Project Content (continued)	■ Visible: Leave this field as is (do not change). By leaving it unchanged, the field will remain visible on the form, but its behavior will be controlled by the Read-only setting. 4. Submit the Action Click Submit to save the UI Policy Action. The action is now linked to the High Impact Control UI Policy and will automatically apply to the Urgency field when an Incident has High Impact. 5. Optional Verification To test the action: ■ Open an Incident record. ■ Set the Impact field to High. ■ Check that the Urgency field becomes read-only. ■ Change Impact to a different value to confirm that the field reverts back if needed. **Task 3: Create onChange Client Script** **Activity 1: Create Client Script**	

S.No.	Parameter	Values	Screenshot

6	Project Content (continued)	1. Navigate to Client Scripts:
		Go to System UI → Client Scripts.
		2. Click New to create a new Client Script.
		3. Fill in the details:
		■ Name: Auto set urgency for high impact – clearly shows the purpose. ■ Table: Incident – the script runs only on Incident records.
		■ Type: onChange – executes when a field value changes.
		■ Field name: Impact – triggers the script when Impact is updated.
		■ Active: true – ensures the script runs immediately.
		4. Paste the script in the script editor section.
		5. Click Submit to save the Client Script.
		6. Verify: Open an Incident record and change the Impact field to High. The Urgency field
		should update automatically
		Script:
		`function onChange(control, oldValue, newValue, isLoading) {`
		`if(isLoading || newValue == '') {`
		return;
		`}`
		`if (newValue == '1') {`
		`g_form.setValue('urgency', '1');`
		`g_form.addInfoMessage('Urgency set to High for High impact incident.');`
		`}`	

S.No.	Parameter	Values	Screenshot
7	Project Content (continued)	} 5. Click Submit. **Task 4: Create onSubmit Client Script (Save Validation)** **Activity 1: Create Client Script**	

S.No.	Parameter	Values	Screenshot

8	Project Content (continued)	1. Navigate to Client Scripts:
		Go to System UI → Client Scripts in your ServiceNow instance.
		2. Click New to create a new Client Script.
		3. Fill in the details:
		■ Name: Prevent save if Assigned To missing – clearly indicates that the script
		prevents saving if the field is empty.
		■ Table: Incident – applies the script only to Incident records.
		■ Type: onSubmit – executes when the user tries to save the record.
		■ Active: true – ensures the script is enforced immediately.
		4. Paste the script into the script editor section. This script should check if the Assigned To
		field is empty and prevent the record from saving if it is.
		5. Click Submit to save the Client Script.
		6. Verify: Try saving an Incident without filling the Assigned To field. The script should
		prevent the save and display a warning.
		7. Paste the script:
		`function onSubmit() {`
		`if (g_form.getValue('impact') == '1' &&`
		`g_form.getValue('assigned_to') == '') {`
		`g_form.showErrorBox(`
		'assigned_to', 'Assigned To is mandatory for High impact incidents.'
		`);`	

S.No.	Parameter	Values	Screenshot

9	Project Content (continued)	`return false;`
		`}`
		`return true;`
		`}`
		8. Click Submit.
		Task 5: Create onCellEdit Client Script
		Activity 1: Create Client Script
		1. Navigate to Client Scripts:
		Go to System UI → Client Scripts in your ServiceNow instance.
		2. Click New to create a new Client Script.
		3. Fill in the details:	

S.No.	Parameter	Values	Screenshot

10	Project Content (continued)	■ Name: Prevent state change via list edit – clearly indicates that the script prevents
		users from changing the State field directly in a list.
		■ Table: Incident – applies the script only to Incident records.
		■ Type: onCellEdit – runs whenever a field is edited directly in a list view.
		■ Field name: State – the script will monitor changes to the State field.
		■ Active: true – ensures the script is enforced immediately.
		4. Paste the script into the script editor section. The script should prevent unauthorized
		changes to the State field when editing from the list view.
		5. Click Submit to save the Client Script.
		6. Verify: Open an Incident list and try to change the State field directly in the list. The
		script should block the change as intended.
		7. In the Script field, paste the following code:
		`function onCellEdit(sysIDs, table, oldValues, newValue, callback)`
		`{`
		`alert('State cannot be updated using list editing. Please open the Incident.');`
		`callback(false);`
		`}`
		8. Click Submit.	

S.No.	Parameter	Values	Screenshot
11	Project Content (continued)	**Task 6: Testing the Configuration** **Activity 1: Test Mandatory Enforcement** 1. Navigate to Incident: Go to Incident → Create New to open a new Incident form. 2. Set the Impact Field: Select High for the Impact field. This will trigger the UI Policy and Client Script behaviors you configured earlier. 3. Leave Assigned To Empty: Do not fill in the Assigned To field. This will test the onSubmit Client Script that prevents saving if Assigned To is missing. 4. Click Submit: Attempt to save the Incident record. 5. Verify the Result:	

S.No.	Parameter	Values	Screenshot
12	Project Content (continued)	■ The Incident should not be saved because Assigned To is required when Impact is High. ■ An error message should appear, indicating that the Assigned To field cannot be empty. ■ Confirm that all UI Policy actions, such as read-only fields or automated urgency updates, are working as expected.	

S.No.	Parameter	Values	Screenshot
13	Project Content (continued)	**Activity 2: Test Successful Save** 1. Fill the Assigned To Field: Open the Incident form and select a user in the Assigned To field. This ensures the onSubmit Client Script condition is satisfied. 2. Click Submit: Attempt to save the Incident record. 3. Verify the Result: ■ The Incident should be saved successfully without any errors. ■ Confirm that all other configured behaviors, such as: ■ Urgency auto-setting (from the onChange Client Script) ■ Read-only fields (from the UI Policy Action) ■ State restrictions (from the onCellEdit Client Script) are functioning as expected	

S.No.	Parameter	Values	Screenshot
14	Project Content (continued)	**Activity 3: Reverse Condition Test** 1. Open an Existing Incident: Go to Incident → Open and select an Incident where Impact is currently set to High. 2. Change Impact: Update the Impact field from High to Medium. This will test the Reverse if false functionality of your UI Policy and the behavior of your Client Scripts. 3. Verify the Field Behavior: ■ Assigned To should no longer be mandatory. ■ Urgency should now be editable, meaning the read-only UI Policy action is reversed. 4. Save the Record: Click Submit to save the changes. The record should save successfully without errors, confirming that the reverse condition works as intended.	

S.No.	Parameter	Values	Screenshot
15	Project Content (continued)	**Activity 4: Test List Edit Blocking** 1. Navigate to the Incident List: Go to Incident → All to view all Incident records in a list view. 2. Attempt a List Edit: ■ Locate any Incident in the list. ■ Double-click the State field to try editing it directly from the list. 3. Verify the Behavior: ■ An alert message should appear, indicating that direct changes to the State field are not allowed. ■ The State value should remain unchanged, confirming that the onCellEdit Client Script is working correctly.	

S.No.	Parameter	Values	Screenshot
16	Project Content (continued)	**Activity 5: Test Form-Based Update** 1. Open the Incident Record: Navigate to the same Incident record you used in previous tests. 2. Change the State: On the Incident form, update the State field to a new value. This tests that changes via the form (not the list) are allowed. 3. Click Update: Save the record by clicking Update. 4. Verify the Result: ■ The State change should be saved successfully. ■ Confirm that other fields, such as Assigned To and Urgency, still follow the configured UI Policy and Client Script rules.	

S.No.	Parameter	Values	Screenshot
17	Project Content (continued)	**Conclusion** TheImplementClient Script & UI Policy (Incident) micro project effectively demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using onChange, onSubmit, and onCellEdit scripts alongside UI Policy actions, the project ensures fields like Assigned To and Urgency respond appropriately to changes in Impact, maintaining clean and	

S.No.	Parameter	Values	Screenshot
18	Project Content (continued)	consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in ServiceNow.	